

PROCESOS DE DESARROLLO II

Conceptos del diseño de software

Clase 2 — Principios que guían un buen diseño

¿Qué es el diseño?

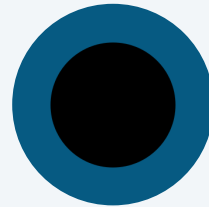
El diseño integra tecnología y personas alrededor de un propósito humano

Marco Vitruvio, crítico romano de arquitectura, decía que un buen edificio necesitaba tres cualidades. Lo mismo aplica al buen software.



Resistencia

Un programa no debe tener errores que impidan su funcionamiento.



Funcionalidad

Debe ser apropiado para los fines que persigue.



Belleza

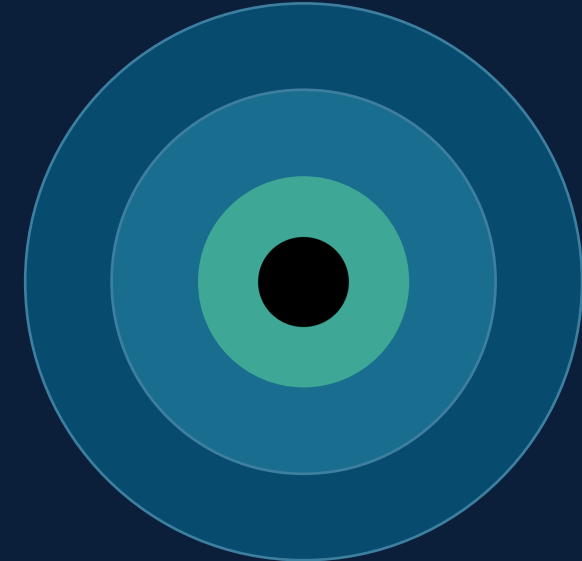
La experiencia de usarlo debe ser placentera.

Según Roger Pressman

El objetivo del diseño es aplicar principios, conceptos y prácticas que llevan a un sistema de alta calidad — un modelo que implante correctamente los requerimientos y satisfaga a quien lo usa.

El proceso va de una visión panorámica (arquitectura) hacia el detalle: se definen subsistemas, se establece cómo se comunican, se identifican componentes y se diseñan las interfaces externa, interna y de usuario.

De lo panorámico al detalle



Arquitectura

la visión panorámica



Subsistemas

cómo se comunican entre sí

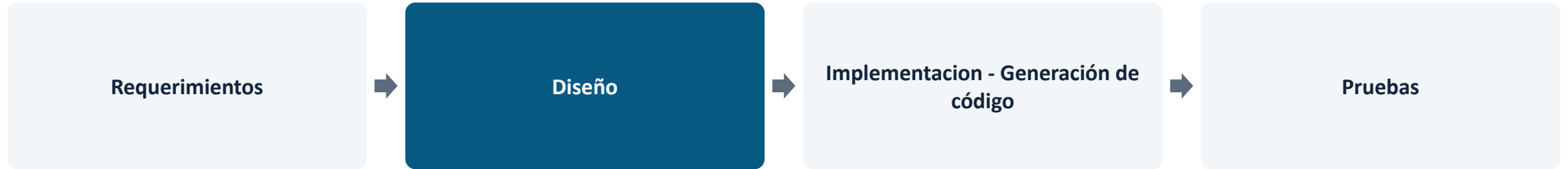


Componentes e interfaces

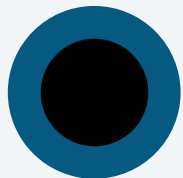
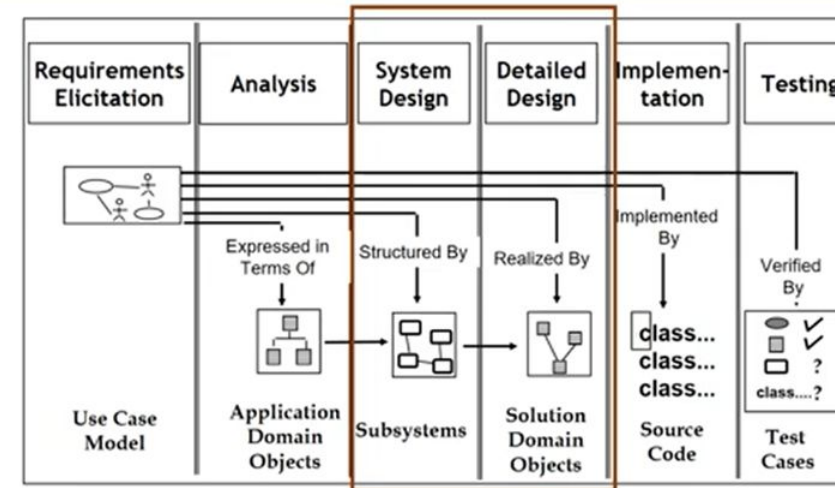
el mayor nivel de detalle

¿Dónde se ubica el diseño?

Primera de las tres actividades técnicas, una vez definidos los requerimientos



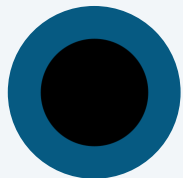
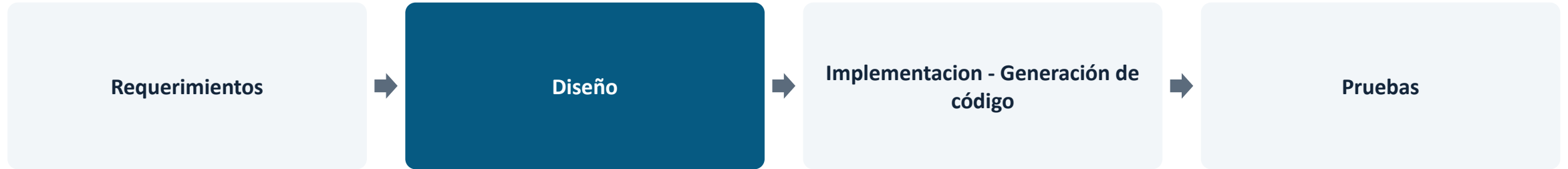
Diseñar, generar código y probar son las tres actividades que construyen y verifican el software una vez analizados y especificados los requisitos.



El diseño del software está en el núcleo técnico de la Ingeniería de Software, independientemente del modelo de proceso que se utilice.

Ciclo de vida de un Sistema de software

Primera de las tres actividades técnicas, una vez definidos los requerimientos



El diseño del software está en el núcleo técnico de la Ingeniería de Software, independientemente del modelo de proceso que se utilice.

Conceptos clave del diseño

Ocho ideas que Pressman ubica en el centro de un buen diseño de software



Abstracción

Crear componentes reutilizables



Arquitectura

Entender la estructura general



Patrones

Técnica de diseño comprobada



Modularidad

Software entendible y testeable



Ocultamiento de información

Reduce la propagación de errores



Independencia funcional

Módulos eficaces y desacoplados



Refinamiento

Mecanismo de diseño progresivo

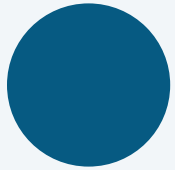


Clases orientadas a objetos

Y sus características asociadas

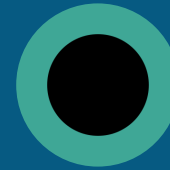
¿Qué es diseñar?

El objetivo es producir un modelo con resistencia, funcionalidad y belleza



1. Diversificación

Adquirir un repertorio de alternativas: componentes, soluciones y conocimiento — contenidos en catálogos, libros de texto y en la propia experiencia.



2. Convergencia

A partir de ese repertorio, seleccionar y combinar las alternativas hasta llegar a la solución que mejor se adapta al proyecto.

Sistemas mal diseñados

Dos ejemplos frecuentes en la práctica profesional



Mala interfaz (viola belleza y usabilidad)

Sistemas de gestión antiguos en organismos públicos: botones sin etiquetas claras, flujos de 5 pantallas para cargar un dato, nada responsivo ni accesible.

Resultado: usuarios frustrados, errores frecuentes, capacitación constante.



Falla de seguridad (viola confiabilidad)

Sistemas web con acceso directo por URL (por ejemplo /usuario?id=2) sin validar roles ni encriptar datos.

Resultado: brechas de seguridad, pérdida de confianza, exposición legal.

Sistemas mal diseñados: escalabilidad

Cuando no se planifica para el crecimiento o los picos de uso



Picos de tráfico no planificados

Sistemas de venta de entradas que colapsan en el "on-sale" de un recital masivo: no se diseñó para la concurrencia real esperada.

Resultado: caídas del sitio, ventas perdidas, reputación dañada.



Trámites de último momento

Portales de trámites gubernamentales que se caen justo el último día de un vencimiento impositivo, cuando más se necesitan.

Resultado: usuarios que no llegan a cumplir el plazo, colapso de canales alternativos.

Mal diseño modular

Viola modularidad y ocultamiento de información



Lógica de negocio dispersa

El mismo código se repite en distintos componentes. Todo está acoplado: modificar un módulo rompe otros.

Impacto: mantenimiento difícil, bajo rendimiento del equipo, errores ocultos.



Cómo se ve en el código

- Todo el código en una única clase o pocos archivos gigantes.
- UI, lógica de negocio y acceso a datos mezclados en un mismo lugar.
- No hay separación en capas (presentación, lógica, datos).
- Cambiar el cálculo de una calificación obliga a tocar el archivo de login.

Complejidad accidental

Cuando el propio diseño agrega dificultad que el problema no pedía



Configuración excesiva

Decenas de opciones y checkboxes que reflejan indecisión de diseño más que una necesidad real del usuario.

Impacto: el usuario no sabe qué elegir, y cada opción es una rama más para mantener y probar.



Sobre-ingeniería

- Capas de abstracción agregadas "por las dudas", sin necesidad real.
- Entender un flujo simple exige saltar por varios archivos o servicios.
- El código termina siendo más difícil de mantener que el problema que resolvía.

El diseño se centra en cuatro áreas

Toda representación de ingeniería de lo que vamos a construir pasa por ellas



Diseño de Datos

Las estructuras que van a sostener la información del sistema



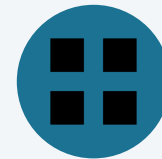
Diseño de la Arquitectura

La visión panorámica de cómo se organiza el sistema



Diseño de Interfaces

Externa, interna y de usuario

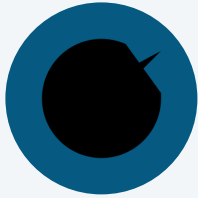


Diseño de Componentes

El detalle necesario para implementar cada parte

Atributos de calidad que surgen: CI/CD

Lo que exige pasar código a producción de forma frecuente y confiable



Testability

Qué tan fácil es automatizar pruebas unitarias y de integración.

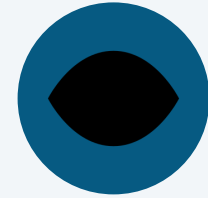
Sin tests automatizados, un pipeline de CI no tiene sentido.



Deployability

Qué tan simple y reversible es poner una versión nueva en producción.

Blue-green deployment, feature flags, rollbacks automáticos.



Observabilidad

Logs, métricas y trazas que muestran qué pasa en producción sin reproducir el error localmente.

Es la base para desplegar seguido con confianza.

Cuando el sistema corre sobre infraestructura elástica y de terceros



Elasticidad

Crece o achicarse agregando instancias, no solo con una máquina más potente.



Resiliencia

El sistema sigue funcionando, aunque degradado, cuando falla una parte.



Costo (cost-efficiency)

Cada decisión de diseño tiene un costo medible: se paga por lo que se usa.



Portabilidad entre nubes

Qué tan atado queda el sistema a un proveedor específico (evitar vendor lock-in).

Gobernanza y automatización

Cómo se gestiona el sistema a lo largo del tiempo, en organizaciones grandes



Trazabilidad / auditabilidad

Poder reconstruir quién cambió qué, cuándo y por qué.

Crítico en sistemas regulados: bancos, salud.



Cumplimiento normativo

Adherencia a normas como GDPR, PCI-DSS o regulaciones locales de datos.

Cada vez más exigido por ley, no solo por buena práctica.



Gobernanza de APIs

Versionado y contratos claros entre equipos que consumen y proveen servicios.

Clave en arquitecturas de microservicios.



Automatización

Principio transversal: todo lo que se repite debería ejecutarse sin intervención manual.

Tests, builds, despliegues, provisioning de infraestructura.

Atributos de calidad

Atributos clásicos	Atributos arquitectónicos modernos	Atributos organizacionales	Atributos asociados a IA
Cohesión Acoplamiento Reutilización Mantenibilidad Extensibilidad Flexibilidad Testabilidad	Escalabilidad Resiliencia Disponibilidad Observabilidad Desplegabilidad Evolucionabilidad Seguridad por diseño	Gobernanza Automatización Auditabilidad Portabilidad Cloud Sostenibilidad	Diseño asistido por IA Validación humana Trazabilidad de decisiones Gobernanza de IA

El buen diseño de software ya no sólo debe producir sistemas correctos y mantenibles; también debe facilitar su construcción, despliegue, operación, evolución y gobierno en entornos distribuidos y asistidos por Inteligencia Artificial.

¿Quiénes lo hacen? ¿Por qué importa?

Roles dentro del equipo de ingeniería de software



Ingenieros del software

- Arquitectos de software
- Arquitectos de hardware (despliegue)
- Diseñadores de sistemas
- Diseñadores de interfaces
- Diseñadores de base de datos



¿Por qué es importante?

Necesitamos un plano.

¿Cuáles son los pasos?

Del modelo de requerimientos a cuatro niveles de detalle de diseño

Arquitectura del sistema

Estructura de datos

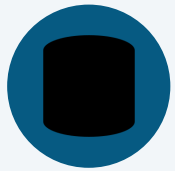
Representación del interfaz

Detalles a nivel de componentes

Se hace para obtener una alta calidad de software.

¿Qué producto obtenemos?

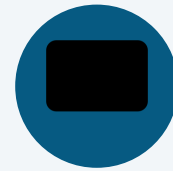
Una especificación de diseño compuesta por cuatro modelos



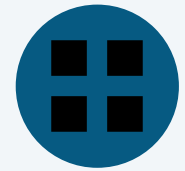
Modelo de datos



Modelo de arquitectura



Modelo de interfaces

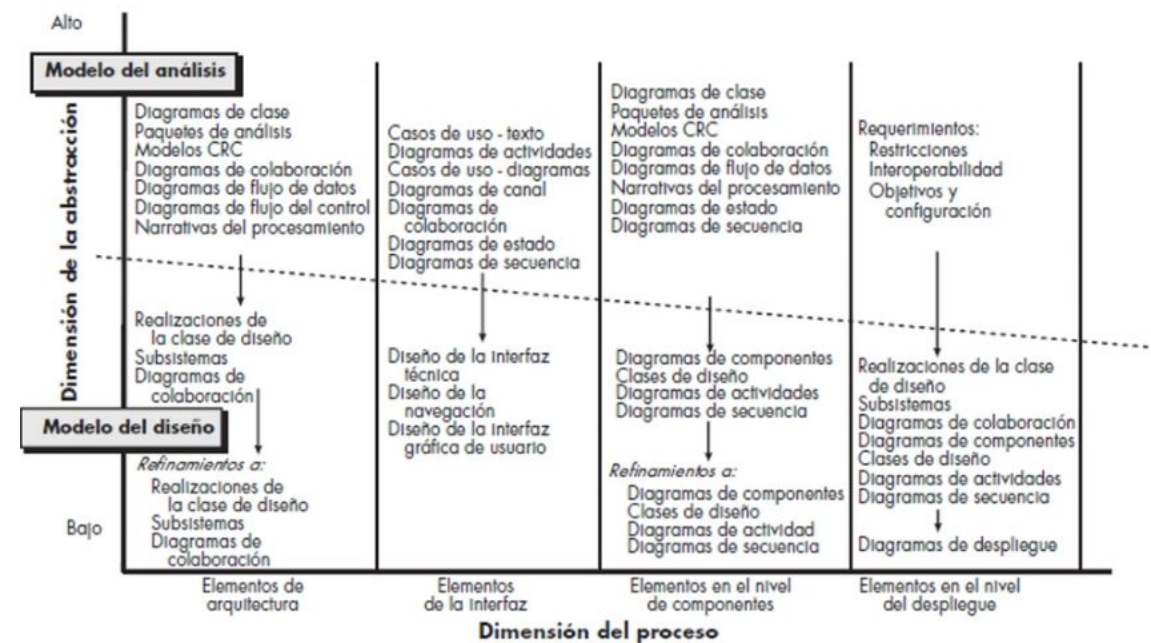
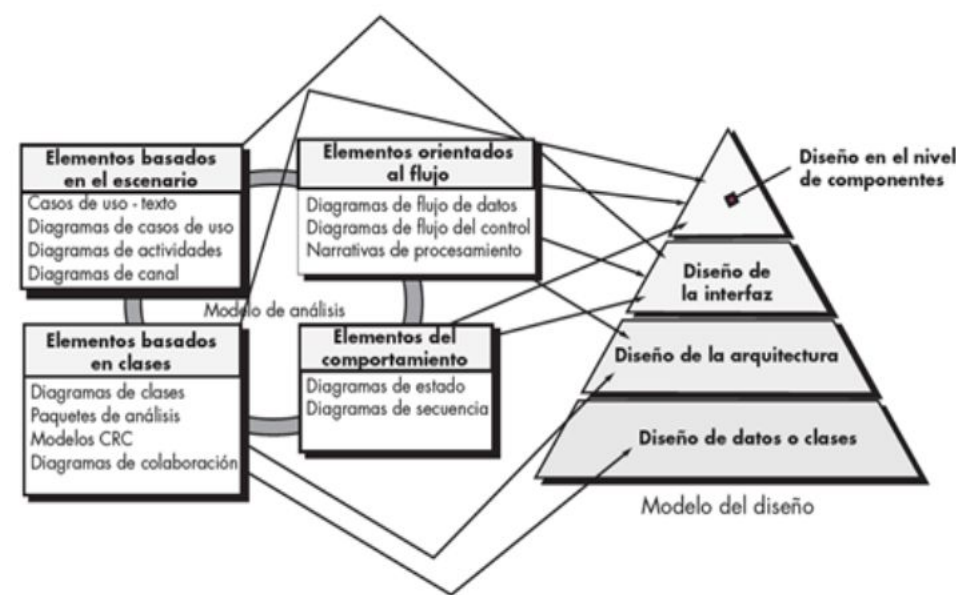


Modelo de componentes

Estos cuatro modelos, en conjunto, describen los datos, la arquitectura, las interfaces y los componentes del sistema.

¿Qué artefactos usamos?

Una especificación de diseño compuesta por cuatro modelos



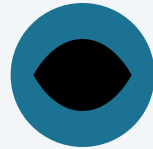
Estos cuatro modelos, en conjunto, describen los datos, la arquitectura, las interfaces y los componentes del sistema.

El enfoque depende del sistema

Según el tipo de sistema, el diseño pone el foco en distintos aspectos



En el usuario



En la accesibilidad



En la información que circula



En la performance



En la seguridad

Evaluando el diseño

Tres características que funcionan como guía para un buen diseño



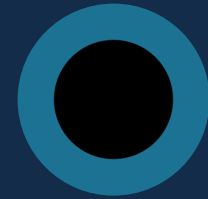
Completo

Implementa todos los requerimientos explícitos y da cabida a los implícitos que esperan los participantes.



Legible y comprensible

Es una guía clara para quienes generan el código, lo prueban y dan soporte después.



Panorama completo

Aborda los dominios de datos, funciones y comportamiento desde la implementación.

Lineamientos para el diseño

Ocho condiciones que en conjunto guían un buen proceso de diseño



Arquitectura con estilos y patrones reconocibles



Modularidad: dividido en elementos o subsistemas



Distintas representaciones: datos, arquitectura, interfaces, componentes



Estructuras de datos basadas en patrones reconocibles



Componentes con independencia funcional



Interfaces que reducen la complejidad de las conexiones



Método repetible, guiado por el análisis de requerimientos



Notación que comunique con eficacia su significado

Estos lineamientos se logran con principios de diseño, una metodología sistemática y revisión constante.